LABSHEET-7

1.(Easy) Write a Java program that creates a thread by extending the Thread class and prints your name 5 times with a 200 ms delay between each print.

```
class MyThread extends Thread {
   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println("Varnika");
         try {
            Thread.sleep(200);
         } catch (InterruptedException e) {
            System.out.println(e);
         }
      }
   }
}


public class Main {
   public static void main(String[] args) {
      MyThread t = new MyThread();
      t.start();
   }
}
```

Output:

Varnika

Varnika

Varnika

Varnika

Varnika

2. (Easy) Rewrite Question 1 using the Runnable interface instead of extending Thread.

```
class MyThread implements Runnable {
   public void run() {
      for (int i = 1; i <= 5; i++) {
         System.out.println("Varnika");


         try {
            Thread.sleep(200);
         } catch (InterruptedException e) {
            System.out.println(e);
         }
      }
   }
}


public class Main {
   public static void main(String[] args) {
      MyThread obj = new MyThread();


      Thread t = new Thread(obj);
      t.start();
   }
}
```

Output:

Varnika

Varnika

Varnika

Varnika

Varnika

3. (Easy) Write a program that prints the name and priority of the currently executing (main) thread, then changes its name and prints it again.

```
public class Main {
   public static void main(String[] args) {

      Thread t = Thread.currentThread();

      System.out.println("Thread Name: " + t.getName());
      System.out.println("Thread Priority: " + t.getPriority());

      t.setName("MyThread");

      System.out.println("New Thread Name: " + t.getName());
   }
}
```

Output:

Thread Name: main

Thread Priority: 5

New Thread Name: MyThread

4. (Easy) Create two threads that each print their own name and a count from 1 to 5. Run the program three times and note whether the output order changes each time.

```
class MyThread extends Thread {

   MyThread(String name) {
      super(name);
   }
```

```
    public void run() {
        for (int i = 1; i <= 5; i++) {
            System.out.println(getName() + " : " + i);
        }
    }
}

public class Main {
    public static void main(String[] args) {

        MyThread t1 = new MyThread("Thread 1");
        MyThread t2 = new MyThread("Thread 2");

        t1.start();
        t2.start();
    }
}
```

Output (Run 1):

```
Thread 1 : 1
Thread 1 : 2
Thread 2 : 1
Thread 2 : 2
Thread 1 : 3
Thread 2 : 3
Thread 1 : 4
Thread 2 : 4
```

Thread 1 : 5

Thread 2 : 5

Output (Run 2 and Run 3):

The output order may be different each time.

5. (Moderate) Write a program with three threads, each named "Reader", "Writer", and "Logger". Each should sleep for a random time (0-500 ms) before printing its name and a message.

```
class MyThread extends Thread {


  MyThread(String name) {
    super(name);
  }


  public void run() {
    try {
      int time = (int)(Math.random() * 501);


      Thread.sleep(time);


      System.out.println(getName() + " : Thread is running");
    }
    catch (InterruptedException e) {
      System.out.println(e);
    }
  }
}


public class Main {
  public static void main(String[] args) {
```

```
    MyThread t1 = new MyThread("Reader");
    MyThread t2 = new MyThread("Writer");
    MyThread t3 = new MyThread("Logger");

    t1.start();
    t2.start();
    t3.start();
  }
}
```

Output (Example):

```
Logger : Thread is running
Reader : Thread is running
Writer : Thread is running
```

6. (Easy) Write a program that demonstrates the use of isAlive() to check a thread's status before it starts, while it runs, and after it finishes.

```
class MyThread extends Thread {

  MyThread(String name) {
    super(name);
  }

  public void run() {
    try {
      int time = (int)(Math.random() * 501);

      Thread.sleep(time);
```

```
        System.out.println(getName() + " : Thread is running");
      }
      catch (InterruptedException e) {
        System.out.println(e);
      }
    }
}

public class Main {
   public static void main(String[] args) {

      MyThread t1 = new MyThread("Reader");
      MyThread t2 = new MyThread("Writer");
      MyThread t3 = new MyThread("Logger");

      t1.start();
      t2.start();
      t3.start();
   }
}
```

Output (Example):

Logger : Thread is running

Reader : Thread is running

Writer : Thread is running

7. (Moderate) Create a thread that performs a long-running task (a loop summing numbers from 1 to 10,000). Use join() in the main method to ensure the main thread prints the "Task finished" message only after the calculation completes.

```
class MyThread extends Thread {
```

```
    public void run() {
        int sum = 0;

        for (int i = 1; i <= 10000; i++) {
            sum = sum + i;
        }

        System.out.println("Sum = " + sum);
    }
}

public class Main {
    public static void main(String[] args) throws InterruptedException {

        MyThread t = new MyThread();

        t.start();

        t.join();

        System.out.println("Task finished");
    }
}
```

Output:

```
Sum = 50005000
Task finished
```

8. (Easy) Write a program that creates a thread, sets its priority to Thread.MAX_PRIORITY, and another thread with Thread.MIN_PRIORITY. Print each thread's priority before starting them.

```
class MyThread extends Thread {

  MyThread(String name) {
    super(name);
  }

  public void run() {
    System.out.println(getName() + " is running");
  }
}

public class Main {
  public static void main(String[] args) {

    MyThread t1 = new MyThread("High Priority Thread");
    MyThread t2 = new MyThread("Low Priority Thread");

    t1.setPriority(Thread.MAX_PRIORITY);
    t2.setPriority(Thread.MIN_PRIORITY);

    System.out.println(t1.getName() + " Priority: " + t1.getPriority());
    System.out.println(t2.getName() + " Priority: " + t2.getPriority());

    t1.start();
    t2.start();
  }
```

}

Output:

High Priority Thread Priority: 10

Low Priority Thread Priority: 1

High Priority Thread is running

Low Priority Thread is running

9. (Moderate) Create five threads, assign them priorities 1 through 10 in steps of roughly 2 (or any values you choose), start all of them, and observe (over multiple runs) whether higher priority threads tend to finish their print statements earlier.

```
class MyThread extends Thread {


  MyThread(String name) {
    super(name);
  }


  public void run() {
    System.out.println(getName() +
        " Priority: " + getPriority());
  }
}


public class Main {
  public static void main(String[] args) {


    MyThread t1 = new MyThread("Thread 1");
    MyThread t2 = new MyThread("Thread 2");
    MyThread t3 = new MyThread("Thread 3");
    MyThread t4 = new MyThread("Thread 4");
```

```
        MyThread t5 = new MyThread("Thread 5");

        t1.setPriority(1);
        t2.setPriority(3);
        t3.setPriority(5);
        t4.setPriority(7);
        t5.setPriority(10);

        t1.start();
        t2.start();
        t3.start();
        t4.start();
        t5.start();
    }
}
```

Output (Example):

Thread 5 Priority: 10

Thread 3 Priority: 5

Thread 4 Priority: 7

Thread 2 Priority: 3

Thread 1 Priority: 1

10. (Moderate) A shared BankAccount class has a balance field and a withdraw(int amount) method (not synchronized). Create two threads that each try to withdraw money from the same account 1000 times. Run it and show that the final balance is sometimes incorrect due to a race condition.

```
class BankAccount {
    int balance = 10000;

    void withdraw(int amount) {
```

```
      if (balance >= amount) {
         balance = balance - amount;
      }
   }
}

class MyThread extends Thread {
   BankAccount account;

   MyThread(BankAccount account) {
      this.account = account;
   }

   public void run() {
      for (int i = 1; i <= 1000; i++) {
         account.withdraw(1);
      }
   }
}

public class Main {
   public static void main(String[] args)
         throws InterruptedException {

      BankAccount account = new BankAccount();

      MyThread t1 = new MyThread(account);
```

```
        MyThread t2 = new MyThread(account);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

Output (Expected):

Final Balance: 8000

11. (Moderate) Fix Question 10 by making withdraw() a synchronized method, and confirm the final balance is now always correct.

```
class BankAccount {
    int balance = 10000;

    synchronized void withdraw(int amount) {
        if (balance >= amount) {
            balance = balance - amount;
        }
    }
}

class MyThread extends Thread {
    BankAccount account;
```

```
    MyThread(BankAccount account) {
        this.account = account;
    }

    public void run() {
        for (int i = 1; i <= 1000; i++) {
            account.withdraw(1);
        }
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {

        BankAccount account = new BankAccount();

        MyThread t1 = new MyThread(account);
        MyThread t2 = new MyThread(account);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
```

```
    }
}
```

Output:

Final Balance: 8000

12. (Moderate) Rewrite Question 11 so that instead of synchronizing the whole method, you use a synchronized block that locks only a dedicated lock object, and only around the statements that read/update the balance.

```
class BankAccount {
    int balance = 10000;

    private final Object lock = new Object();

    void withdraw(int amount) {
        synchronized (lock) {
            if (balance >= amount) {
                balance = balance - amount;
            }
        }
    }
}

class MyThread extends Thread {
    BankAccount account;

    MyThread(BankAccount account) {
        this.account = account;
    }
```

```
    public void run() {
        for (int i = 1; i <= 1000; i++) {
            account.withdraw(1);
        }
    }
}

public class Main {
    public static void main(String[] args)
            throws InterruptedException {

        BankAccount account = new BankAccount();

        MyThread t1 = new MyThread(account);
        MyThread t2 = new MyThread(account);

        t1.start();
        t2.start();

        t1.join();
        t2.join();

        System.out.println("Final Balance: " + account.balance);
    }
}
```

Output:

```
Final Balance: 8000
```

13. (Moderate) Implement a simple producer-consumer scenario using wait() and notify() where a producer thread adds numbers (1 to 10) one at a time to a shared single-slot buffer, and a consumer thread reads and prints each number as it becomes available.

```
class Buffer {
  int data;
  boolean available = false;

  synchronized void produce(int value)
      throws InterruptedException {

    while (available) {
      wait();
    }

    data = value;
    System.out.println("Produced: " + data);

    available = true;
    notify();
  }

  synchronized void consume()
      throws InterruptedException {

    while (!available) {
      wait();
    }
```

```
      System.out.println("Consumed: " + data);

      available = false;
      notify();
    }
}

class Producer extends Thread {
    Buffer b;

    Producer(Buffer b) {
      this.b = b;
    }

    public void run() {
      try {
        for (int i = 1; i <= 10; i++) {
          b.produce(i);
        }
      } catch (InterruptedException e) {
        System.out.println(e);
      }
    }
}

class Consumer extends Thread {
    Buffer b;
```

```
    Consumer(Buffer b) {
        this.b = b;
    }

    public void run() {
        try {
            for (int i = 1; i <= 10; i++) {
                b.consume();
            }
        } catch (InterruptedException e) {
            System.out.println(e);
        }
    }
}

public class Main {
    public static void main(String[] args) {

        Buffer b = new Buffer();

        Producer p = new Producer(b);
        Consumer c = new Consumer(b);

        p.start();
        c.start();
    }
```

}

Output:

Produced: 1

Consumed: 1

Produced: 2

Consumed: 2

Produced: 3

Consumed: 3

Produced: 4

Consumed: 4

Produced: 5

Consumed: 5

Produced: 6

Consumed: 6

Produced: 7

Consumed: 7

Produced: 8

Consumed: 8

Produced: 9

Consumed: 9

Produced: 10

Consumed: 10

14. (Moderate) Write a program that creates a deadlock situation using two shared objects, Lock1 and Lock2, where two threads try to acquire them in opposite order. Run it and observe that the program never terminates (you will need to stop it manually).

```
public class Main {


   static Object Lock1 = new Object();
```

```
static Object Lock2 = new Object();

public static void main(String[] args) {

  Thread t1 = new Thread(() -> {
    synchronized (Lock1) {
      System.out.println("Thread 1: Locked Lock1");

      try {
        Thread.sleep(100);
      } catch (Exception e) {
        System.out.println(e);
      }

      synchronized (Lock2) {
        System.out.println("Thread 1: Locked Lock2");
      }
    }
  });

  Thread t2 = new Thread(() -> {
    synchronized (Lock2) {
      System.out.println("Thread 2: Locked Lock2");

      try {
        Thread.sleep(100);
      } catch (Exception e) {
```

```
            System.out.println(e);
        }

        synchronized (Lock1) {
            System.out.println("Thread 2: Locked Lock1");
        }
      }
    });

    t1.start();
    t2.start();
  }
}
```

Output (Example):

Thread 1: Locked Lock1

Thread 2: Locked Lock2

The program does not terminate because both threads are waiting for each other's lock.

15. (Moderate) Fix your deadlock program from Question 14 by ensuring both threads acquire Lock1 and Lock2 in the same order, and confirm the program now completes normally.

```
public class Main {

  static Object Lock1 = new Object();
  static Object Lock2 = new Object();

  public static void main(String[] args) {

    Thread t1 = new Thread(() -> {
      synchronized (Lock1) {
```

```
            System.out.println("Thread 1: Locked Lock1");

            synchronized (Lock2) {
                System.out.println("Thread 1: Locked Lock2");
            }
        }
    });

    Thread t2 = new Thread(() -> {
        synchronized (Lock1) {
            System.out.println("Thread 2: Locked Lock1");

            synchronized (Lock2) {
                System.out.println("Thread 2: Locked Lock2");
            }
        }
    });

    t1.start();
    t2.start();
  }
}
```

Output (Example):

```
Thread 1: Locked Lock1
Thread 1: Locked Lock2
Thread 2: Locked Lock1
Thread 2: Locked Lock2
```